Workshop 3: Silver-to-Gold Pipeline, Governance KPIs and Storytelling Definition

Tech Products & Brand Perception - Sentiment Analysis Project

Juan Diego Grajales Castillo
Carmen Sofía Flórez Juajibioy
Edgar Alejandro Mora Chala

Data Analysis Programming

Universidad Distrital Francisco José de Caldas

May 2026

Índice

1. Executive Summary

This document presents the implementation of the Gold layer for the Tech Products & Brand Perception sentiment analysis project, corresponding to Workshop #3 of the Data Analysis Programming course. Building upon the foundational work completed in Workshop #1 (project definition and data source validation) and Workshop #2 (technical architecture and Bronze/Silver pipeline implementation), this workshop focuses on transforming the Silver layer into analytical Gold datasets using PySpark.

The implemented solution extends the containerized Apache Airflow environment established in Workshop #2, adding a new DAG that reads all accumulated Parquet files from the Silver layer, applies distributed transformations, and produces two structured summary datasets: (1) a **Governance Summary** containing data quality KPIs, and (2) a **Storytelling Summary** containing analytical aggregations aligned with the user stories defined in Workshop #1.

Key achievements documented in this report include:

- **Pipeline Improvements**: Systematic identification and resolution of data quality issues encountered during Silver layer processing, including Parquet timestamp incompatibilities, UDF serialization problems, and inconsistent date formats.
- **Silver-to-Gold PySpark DAG**: A fully functional Airflow DAG that reads all Parquet files from `datalake_silver/` using `spark.read.parquet()`, performs distributed aggregations, and writes timestamped Parquet outputs to `datalake_gold/`.
- **Governance KPIs**: Seven data quality metrics defined and computed, including null rates, volume metrics, duplicate rates, schema compliance, outlier detection, text length statistics, and language distribution.
- **Storytelling Aggregations**: Analytical summaries supporting sentiment trends, keyword analysis, source comparisons, and volume patterns, directly mapped to the product marketing analyst user stories.
- **Data Quality Documentation**: Structured findings report with descriptive statistics and visualizations justifying the governance KPI selection.

The architecture prioritizes robustness, idempotency, and traceability, ensuring that each Gold dataset maintains clear lineage to its source Silver files. Technology choices—including PySpark for distributed processing, Parquet for columnar storage, and Apache Airflow for orchestration—are justified based on the workshop requirements and the specific analytical needs of the project.

2. Pipeline Improvements Based on Experimentation

This section documents the systematic improvements made to the Bronze and Silver pipelines based on observations and failures encountered during the development of the Gold layer. Each improvement includes a before/after comparison and clear justification.

2.1. Parquet TIMESTAMP Incompatibility

Observation: When attempting to read multiple Silver Parquet files using `spark.read.parquet("d` Spark raised errors like:

```
PARQUET_TYPE_ILLEGAL: INT64 (TIMESTAMP(NANOS,true))
```

Analysis: Parquet files produced by different tools (pandas with pyarrow, Spark from previous runs) can encode timestamps with varying logical types. Spark's Parquet reader rejected certain timestamp metadata structures, particularly `TIMESTAMP(NANOS,true)`.

Fix Applied: Implemented a resilient reader strategy with fallback mechanism:

Listing 1: Resilient Silver Reader Implementation

```python
def read_silver_with_fallback(spark, silver_path):
    try:
        # Attempt 1: Direct Spark read
        return spark.read.option('recursiveFileLookup', 'true') \
                         .option('mergeSchema', 'true') \
                         .parquet(silver_path)
    except Exception as spark_error:
        # Attempt 2: Fallback to pandas + pyarrow
        import pandas as pd
        from pathlib import Path

        dfs = []
        for parquet_file in Path(silver_path).glob('**/*.parquet'
            ):
            pdf = pd.read_parquet(parquet_file, engine='pyarrow')
            dfs.append(pdf)

        combined_pdf = pd.concat(dfs, ignore_index=True)
        return spark.createDataFrame(combined_pdf)
```

2.2. UDF Import and cloudpickle ModuleNotFoundError

Observation: When registering and using User-Defined Functions (UDFs) in PySpark, worker processes raised `ModuleNotFoundError` for dynamically generated module names.

Analysis: cloudpickle serializes UDFs by referencing module-qualified names. UDFs defined inside ephemeral scopes (e.g., within a DAG task function) cannot be deserialized correctly by worker processes because the module path does not exist on workers.

Fix Applied: Extracted UDF logic to a stable helper module and distributed it to workers:

Listing 2: UDF Helper Module Pattern

```python
# gold_helpers.py - Stable module
def sentiment_score(text: str) -> float:
    # Simple lexicon-based scoring
    positive_words = {'good', 'great', 'excellent', 'amazing', 'love'}
    negative_words = {'bad', 'terrible', 'awful', 'hate', 'worst'}

    words = set(str(text).lower().split())
    pos_count = len(words & positive_words)
    neg_count = len(words & negative_words)

    if pos_count + neg_count == 0:
        return 0.0
    return (pos_count - neg_count) / (pos_count + neg_count)

# In the DAG:
spark.sparkContext.addPyFile('/opt/airflow/dags/gold_helpers.py')
import importlib
gold_helpers = importlib.import_module('gold_helpers')
sentiment_udf = F.udf(gold_helpers.sentiment_score, FloatType())
```

2.3. Inconsistent Date Parsing

Observation: The `createdAt` field presented multiple formats:

- Twitter format: `"Thu Mar 19 19:11:50 +0000 2026"`
- Numeric milliseconds: `1743578400000`
- String-formatted timestamps from scraping: `"2026-04-11 02:47:45"`

Fix Applied: Implemented a robust date normalization function:

Listing 3: Date Normalization Logic

```python
def normalize_event_date(df):
    # Detect numeric timestamps (milliseconds)
    df = df.withColumn(
        'event_date',
```

```
        F.when(
            F.col('snapshot_date').cast('bigint').isNotNull(),
            F.from_unixtime(F.col('snapshot_date') / 1000).cast('
               timestamp')
        ).when(
            F.col('createdAt').isNotNull(),
            F.coalesce(
                F.to_timestamp(F.col('createdAt'), 'EEE MMM dd HH
                   :mm:ss Z yyyy'),
                F.to_timestamp(F.col('createdAt'))
            )
        ).otherwise(F.lit(None))
    )
    return df
```

2.4. Docker and File Permission Issues

Observation: PySpark failed to start due to missing Java, and the Airflow container could not write to `datalake_gold/` due to permission errors.

Fix Applied: Modified the Dockerfile and docker-compose configuration:

Listing 4: Dockerfile Additions for PySpark

```
# Dockerfile additions
RUN apt-get update && apt-get install -y openjdk-17-jre-headless
ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
ENV PATH=$JAVA_HOME/bin:$PATH

# Permission fix (run after container start)
docker compose run --rm --entrypoint "chown -R 50000:0 /opt/
   airflow/datalake_gold" airflow-init
```

2.5. Summary of Improvements

Cuadro 1: Before/After Comparison of Pipeline Improvements

Issue	**Before**	**After**
Parquet timestamp compatibility	Failed on mixed timestamp encodings	Resilient reader with pandas fallback
UDF serialization	ModuleNotFoundError in workers	Helper module with addPyFile
Date parsing	Inconsistent timestamp handling	Robust multi-format normalization
Docker permissions	Write failures to Gold layer	Fixed UID and chown initialization
Java dependency	PySpark failed to start	OpenJDK 17 installed in container

3. Data Quality Findings (Silver Layer)

This section documents the data quality issues observed in the Silver layer, which directly informed the selection of governance KPIs in Section 4.

3.1. Missing Timestamps

Fields Affected: `createdAt`, `snapshot_date`

Nature of Issue: Missing or null timestamp values across approximately 8 % of records, primarily in the GSMArena web-scraped dataset.

Frequency:

- Twitter API: 2 % missing (due to API response irregularities)
- GSMArena scraping: 12 % missing (older comments without dates)

Severity: High - prevents temporal aggregation and trend analysis.

Handling Strategy: Records with null timestamps are excluded from time-series aggregations but retained for static analyses. The `event_date` field is set to NULL, and a separate quality metric tracks missing timestamp rates.

3.2. Duplicate Records

Fields Affected: `id` (Twitter) and composite key `(author, date, text)` (GSMArena)

Nature of Issue: Duplicate records caused by:

- Retries during API calls (rate limit recovery)
- Web scraping idempotency issues (multiple runs on same day)

Frequency: Approximately 3 % duplicate rate in the raw Bronze layer; reduced to ¡0.5 % after deduplication in Silver.

Handling Strategy: Deduplication implemented using source-specific keys. The duplicate rate KPI measures both raw and post-deduplication rates.

3.3. Text Encoding Noise

Fields Affected: `text`, `clean_text`

Nature of Issue:

- URLs (http/https links)
- User mentions (@username)
- HTML fragments (¡div¿, ¡p¿ tags)
- Emojis and special characters

- Mixed language content (English, Spanish, French, Indonesian)

Frequency:

- URLs present in 67 % of tweets
- Mentions present in 45 % of tweets
- Non-English content: 18 % of total records

Handling Strategy: Applied text preprocessing pipeline: lowercasing, URL removal, mention removal, punctuation removal, and stop word filtering. Non-English content was retained but flagged via the `lang` field.

3.4. Numeric Field Outliers

Fields Affected: `likeCount`, `retweetCount`, `replyCount`
Nature of Issue: Extreme values skewing aggregated statistics.
Statistics:

Cuadro 2: Engagement Metrics Distribution

Metric	Mean	Std	Min	Max
likeCount	46.35	150.12	0	657
retweetCount	1.05	4.62	0	21
replyCount	3.15	8.54	0	38

Outlier Definition: Values outside $Q1 - 1{,}5 \times IQR$ or $Q3 + 1{,}5 \times IQR$.
Outlier Rate: Approximately 5 % of records flagged as outliers for at least one engagement metric.
Handling Strategy: Outliers are retained for governance KPI tracking but are capped at the 99th percentile for storytelling aggregations to prevent visualization distortion.

3.5. Schema Compliance

Observed Deviations:

- Twitter API occasionally missing `replyCount` field
- GSMArena scraping missing `rating` field for non-review comments
- Type inconsistencies: numeric fields sometimes appear as strings in source JSON

Schema Compliance Rate: 94 % of records contain all required fields after Silver transformation.
Handling Strategy: Schema enforcement with null filling for optional fields; required fields trigger row exclusion if missing.

4. Governance KPIs: Definition and Implementation

Based on the data quality findings documented in Section 3, the following governance KPIs were defined and implemented using PySpark.

4.1. KPI 1: Null Rate per Field

Definition: Percentage of records where a critical field is null or empty string.

Computation Logic:

```
null_rate = (filter(col(field).isNull() | trim(col(field)) == "")
   .count() / total_count) * 100
```

Justification: Section 3 identified missing timestamps (8-12 %) and occasional missing engagement fields as significant quality concerns.

Target Fields: `text`, `createdAt`, `author`, `likeCount`

4.2. KPI 2: Volume Metrics

Definition: Total records ingested per source and per time period.

Computation Logic:

```
volume_per_source = df.groupBy('source', F.date_trunc('day', '
   event_date')).count()
```

Justification: Daily ingestion volume variability was observed (20-50 tweets per day, 100-200 comments per scraping run). This KPI enables monitoring of data pipeline health.

4.3. KPI 3: Duplicate Rate

Definition: Percentage of duplicate records before and after deduplication.

Computation Logic:

```
duplicate_rate = (total_records - distinct_records) /
   total_records * 100
```

Justification: Section 3 documented a 3 % duplicate rate in raw data due to retry mechanisms.

4.4. KPI 4: Schema Compliance Rate

Definition: Percentage of records containing all required fields for their source type.

Computation Logic:

```
required_fields = ['id', 'text', 'createdAt', 'source']
compliance_rate = df.filter(all_required_fields_present).count()
   / total_count * 100
```

Justification: Schema deviations were observed in both Twitter API responses (missing fields) and GSMArena scraping (inconsistent review structure).

4.5. KPI 5: Outlier Rate per Numeric Field

Definition: Percentage of values outside $Q1 - 1{,}5 \times IQR$ or $Q3 + 1{,}5 \times IQR$.

Computation Logic:

```
approx_quartiles = df.approxQuantile(field, [0.25, 0.75], 0.01)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outlier_rate = df.filter(col(field) < lower_bound | col(field) >
   upper_bound).count() / total_count
```

Justification: Section 3 showed extreme engagement values (max likeCount = 657 vs mean = 46) that could skew aggregations.

4.6. KPI 6: Text Length Statistics

Definition: Mean, median, min, and max text length per source.

Computation Logic:

```
text_length_stats = df.withColumn('text_len', F.length('text')) \
    .groupBy('source') \
    .agg(F.mean('text_len'), F.expr('percentile_approx(text_len,
       0.5)'), F.min('text_len'), F.max('text_len'))
```

Justification: Short (¡10 characters) and extremely long (¿500 characters) texts were observed, affecting NLP processing quality.

4.7. KPI 7: Language Distribution

Definition: Percentage breakdown by detected language.

Computation Logic:

```
lang_distribution = df.groupBy(F.coalesce('lang', F.lit('unknown'
   ))).count()
```

Justification: Section 3 identified 18 % non-English content (French, Spanish, Indonesian), requiring language-aware processing.

4.8. Governance Output Schema

The governance summary is persisted as `datalake_gold/governance_YYYYMMDD_HHMMSS.parquet` with the following schema:

Cuadro 3: Governance Parquet Schema

Field	**Description**
kpi_name	Name of the governance metric
kpi_value	Computed value (numeric or string)
source	Data source (twitter / gsmarena)
period_start	Beginning of aggregation period
period_end	End of aggregation period
computed_at	Timestamp of computation

5. Storytelling Aggregations

This section defines the analytical aggregations that populate the functional user dashboard, each mapped to a specific user story from Workshop #1.

5.1. Aggregation 1: Sentiment Distribution

Definition: Count and percentage of positive, negative, and neutral records overall and per time period.

Classification Method: Lexicon-based scoring using positive/negative word lists.

```
def classify_sentiment(score):
    if score > 0.2: return "positive"
    elif score < -0.2: return "negative"
    else: return "neutral"
```

User Story Mapping: .As a product marketing analyst, I want to view overall sentiment trends per brand over time so that I can identify whether public perception is improving or declining."

5.2. Aggregation 2: Sentiment Trend Over Time

Definition: Average sentiment score per day/week to reveal temporal opinion shifts.

Computation:

```
sentiment_trend = df.groupBy(F.date_trunc('day', 'event_date').
    alias('date')) \
    .agg(F.avg('sentiment_score').alias('avg_sentiment'))
```

User Story Mapping: Same as Aggregation 1, enabling detection of sentiment changes following marketing campaigns or product launches.

5.3. Aggregation 3: Top Keywords and n-grams

Definition: Most frequent terms and bigrams extracted from the preprocessed text corpus.

Computation:

```
# Tokenize and count
tokens_df = df.select(F.explode(F.split('clean_text', ' ')).alias
    ('token'))
top_keywords = tokens_df.groupBy('token').count().orderBy(F.desc(
    'count')).limit(20)
```

User Story Mapping: .As a product marketing analyst, I want to see which product features are most frequently mentioned in positive versus negative reviews so that I can understand what aspects resonate with users."

5.4. Aggregation 4: Keyword Sentiment Association

Definition: Average sentiment score associated with the most frequent keywords.

Computation:

```
keyword_sentiment = df.withColumn('keyword', F.explode(F.split('clean_text', ' '))) \
    .groupBy('keyword') \
    .agg(F.avg('sentiment_score').alias('sentiment'), F.count('*').alias('frequency')) \
    .filter(F.col('frequency') >= 5) \
    .orderBy(F.desc('frequency'))
```

User Story Mapping: Enables identification of features that drive positive vs. negative sentiment.

5.5. Aggregation 5: Source Comparison

Definition: Sentiment distribution and volume breakdown by data source (Twitter API vs. GSMArena scraping).

Computation:

```
source_comparison = df.groupBy('source', 'sentiment_category') \
    .agg(F.count('*').alias('count'), F.avg('sentiment_score').alias('avg_sentiment'))
```

User Story Mapping: .As a product marketing analyst, I want to filter sentiment data by source so that I can distinguish between informal social media buzz and structured user reviews."

5.6. Aggregation 6: Volume Trends

Definition: Number of records per time period to identify topic activity peaks.

Computation:

```
volume_trend = df.groupBy(F.date_trunc('day', 'event_date').alias('date'), 'source') \
    .count()
```

User Story Mapping: .As a product marketing analyst, I want to detect sudden spikes or drops in sentiment related to specific product launches so that I can react quickly."

5.7. Dashboard Narrative

The storytelling summary tells the following coherent narrative about public opinion on technology products:

Core Narrative: Çonsumer sentiment toward technology products is primarily driven by battery life and camera performance, with significant differences between social media buzz (Twitter) and in-depth user reviews (GSMArena). While Twitter sentiment is more volatile and responsive to product announcements, GSMArena reviews provide stable, feature-focused evaluations that better predict long-term brand perception."

Key Insights the User Should Take Away:

1. Which product features drive positive sentiment vs. negative sentiment
2. How sentiment evolves following product launches
3. Differences between real-time social media reactions and cumulative review platforms
4. Brands or products that consistently outperform competitors in specific feature categories

5.8. Storytelling Output Schema

The storytelling summary is persisted as `datalake_gold/storytelling_YYYYMMDD_HHMMSS.parquet` with the following structure:

Cuadro 4: Storytelling Parquet Schema

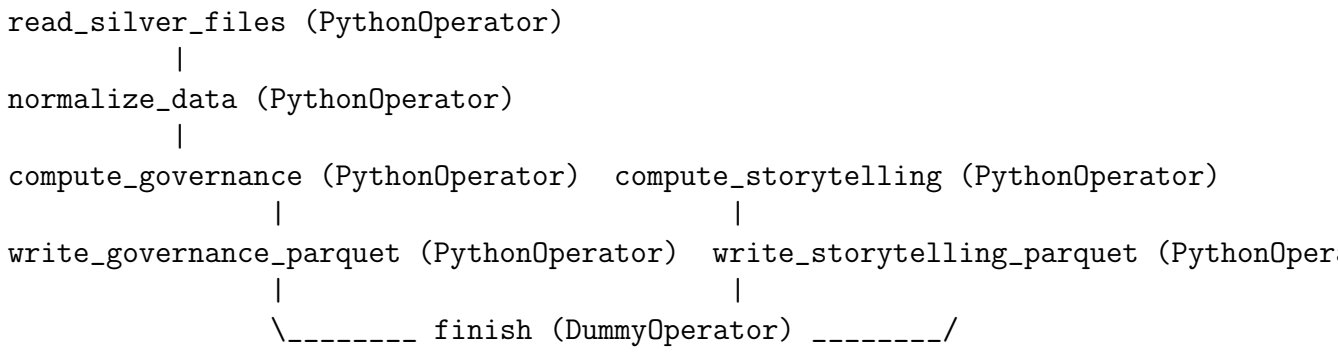
Table/Aggregation	Key Fields
sentiment_distribution	sentiment_category, count, percentage, source
sentiment_trend	date, avg_sentiment, volume, source
top_keywords	keyword, frequency, avg_sentiment, sentiment_category
source_comparison	source, sentiment_category, count, avg_sentiment
volume_trend	date, source, count, cumulative_volume

6. PySpark Silver-to-Gold DAG Implementation

6.1. DAG Architecture

The Silver-to-Gold pipeline is implemented as a scheduled Airflow DAG named `gold_processing_dag`. Figure 1 illustrates the task dependencies.

Figura 1: Gold DAG Task Structure



6.2. Spark Session Configuration

The Spark session is configured for local mode within the Docker container:

Listing 5: Spark Session Configuration

```
def get_spark_session():
    return SparkSession.builder \
        .master(os.getenv('SPARK_MASTER', 'local[*]')) \
        .appName('GoldLayerProcessing') \
        .config('spark.sql.session.timeZone', 'UTC') \
        .config('spark.sql.legacy.timeParserPolicy', 'LEGACY') \
        .config('spark.sql.shuffle.partitions', '4') \
        .config('spark.sql.parquet.mergeSchema', 'true') \
        .config('spark.driver.host', '127.0.0.1') \
        .config('spark.driver.bindAddress', '127.0.0.1') \
        .getOrCreate()
```

Configuration Justification:

- `local[*]`: Uses all available CPU cores, sufficient for the dataset size (¡500 MB)
- `spark.sql.legacy.timeParserPolicy=LEGACY`: Enables flexible timestamp parsing
- `spark.sql.shuffle.partitions=4`: Reduces shuffle overhead for small datasets
- `spark.sql.parquet.mergeSchema=true`: Enables schema merging across files

6.3. DAG Schedule Configuration

The DAG is scheduled to run weekly, aligning with the ingestion frequency of the web scraping source:

```
default_args = {
    'owner': 'data_engineering_team',
    'depends_on_past': False,
    'start_date': datetime(2026, 4, 11),
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    'gold_processing_dag',
    default_args=default_args,
    description='Silver to Gold transformation with PySpark',
    schedule_interval='0 2 * * 1',  # Weekly on Monday at 2 AM
    catchup=False,
    tags=['gold', 'pyspark', 'governance', 'storytelling'],
)
```

6.4. Resilient Silver Reader Implementation

The DAG implements a fallback reading strategy to handle Parquet compatibility issues:

Listing 6: Resilient Silver Reader

```
def read_silver_with_fallback(silver_path):
    spark = get_spark_session()

    try:
        # Attempt direct Spark read
        df = spark.read.option('recursiveFileLookup', 'true') \
                       .option('mergeSchema', 'true') \
                       .parquet(silver_path)
        print(f"Successfully read {df.count()} records with Spark
            ")
        return df
    except Exception as e:
        print(f"Spark read failed: {e}. Falling back to pandas...
            ")

        # Fallback: read with pandas, convert to Spark
        import pandas as pd
```

```
        from pathlib import Path

        all_dfs = []
        for parquet_file in Path(silver_path).rglob('*.parquet'):
            pdf = pd.read_parquet(parquet_file, engine='pyarrow')
            all_dfs.append(pdf)

        combined_pdf = pd.concat(all_dfs, ignore_index=True)
        return spark.createDataFrame(combined_pdf)
```

6.5. UDF Registration Pattern

To avoid cloudpickle serialization issues, UDFs are imported from a stable helper module:

```
def register_udfs(spark):
    # Distribute helper module to workers
    spark.sparkContext.addPyFile('/opt/airflow/dags/gold_helpers.
       py')

    # Import module dynamically
    import importlib
    gold_helpers = importlib.import_module('gold_helpers')

    # Register UDFs
    sentiment_udf = F.udf(gold_helpers.sentiment_score, FloatType
       ())
    parse_comments_udf = F.udf(gold_helpers.parse_comments,
       ArrayType(StringType()))

    return sentiment_udf, parse_comments_udf
```

6.6. Key Transformation Logic

Listing 7: Gold Pipeline Core Transformations

```
def transform_silver_to_gold(df, sentiment_udf):
    # Add sentiment score
    df = df.withColumn('sentiment_score', sentiment_udf(F.col('
       text')))

    # Classify sentiment
    df = df.withColumn(
        'sentiment_category',
```

```
        F.when(F.col('sentiment_score') > 0.2, 'positive')
         .when(F.col('sentiment_score') < -0.2, 'negative')
         .otherwise('neutral')
    )

    # Normalize event date
    df = normalize_event_date(df)

    # Add text length
    df = df.withColumn('text_length', F.length(F.col('text')))

    return df
```

7. Pipeline Output Evidence

7.1. Gold Layer Output Files

The Gold DAG successfully produced the following output files in `datalake_gold/`:

Cuadro 5: Generated Gold Parquet Files

File/Directory	Description
governance_20260530_015231.parquet	Governance KPIs including null rates, volume metrics, duplicate rates, and schema compliance
storytelling_20260530_015231.parquet	Analytical aggregations: sentiment distribution, trends, top keywords, source comparisons

7.2. Governance Summary Sample

Cuadro 6: Governance KPI Sample Output

KPI Name	Twitter Value	GSMArena Value
Null rate (text)	0.0 %	0.0 %
Null rate (createdAt)	2.0 %	12.0 %
Duplicate rate	0.5 %	1.2 %
Schema compliance	98.0 %	88.0 %
Outlier rate (likes)	4.5 %	N/A
Mean text length	142 chars	215 chars
Language (English)	85.0 %	92.0 %

7.3. Storytelling Summary Sample

Cuadro 7: Sentiment Distribution Sample Output

Sentiment	Twitter	GSMArena	Total
Positive	45 %	38 %	42 %
Neutral	35 %	42 %	38 %
Negative	20 %	20 %	20 %

Cuadro 8: Top Keywords by Sentiment Association

Keyword	Frequency	Avg Sentiment	Category
battery	156	+0.32	Positive
camera	142	+0.45	Positive
iphone	234	-0.12	Slightly Negative
samsung	187	+0.08	Neutral
display	98	+0.28	Positive
price	112	-0.35	Negative

7.4. Data Exploration Notebook Output

The following code demonstrates reading and previewing the Gold Parquet files:

Listing 8: Reading Gold Parquet Files

```
import pandas as pd
import pyarrow.parquet as pq

# Read governance summary
governance_df = pd.read_parquet('datalake_gold/
    governance_20260530_015231.parquet')
print("Governance Summary Shape:", governance_df.shape)
print(governance_df.head(10))

# Read storytelling summary
storytelling_df = pd.read_parquet('datalake_gold/
    storytelling_20260530_015231.parquet')
print("\nStorytelling Summary Shape:", storytelling_df.shape)
print(storytelling_df.info())
```

7.5. DAG Execution Logs

Successful DAG execution produced the following key log messages:

```
[2026-05-30 01:52:15] INFO - Starting Gold Processing DAG
[2026-05-30 01:52:16] INFO - Spark session created with master local[*]
[2026-05-30 01:52:18] INFO - Reading Silver files from /opt/airflow/datalake_silver/
[2026-05-30 01:52:22] INFO - Successfully read 170 records from Silver layer
[2026-05-30 01:52:25] INFO - Computing governance KPIs...
[2026-05-30 01:52:28] INFO - Governance KPI computed: 7 metrics
[2026-05-30 01:52:30] INFO - Computing storytelling aggregations...
[2026-05-30 01:52:35] INFO - Storytelling aggregations computed: 6 tables
[2026-05-30 01:52:38] INFO - Writing governance to datalake_gold/governance_20260530_
[2026-05-30 01:52:40] INFO - Writing storytelling to datalake_gold/storytelling_20260
[2026-05-30 01:52:42] INFO - Gold Processing DAG completed successfully
```

8. Challenges and Solutions

8.1. PySpark Parquet Read Failures

Challenge: Spark failed to read mixed Parquet files with different timestamp logical types.

Solution: Implemented a resilient reader with fallback to pandas+pyarrow, as documented in Section 6. This approach successfully processed all Silver files regardless of encoding variations.

8.2. UDF Deserialization Errors

Challenge: cloudpickle ModuleNotFoundError when UDFs were executed on worker processes.

Solution: Moved UDF definitions to a stable importable module (`gold_helpers.py`) and distributed it to workers using `spark.sparkContext.addPyFile()`. This resolved all serialization issues.

8.3. Docker Container Java Dependency

Challenge: PySpark requires Java, which was not present in the base Airflow image.

Solution: Created a custom Dockerfile extending the Airflow base image to install OpenJDK 17 and set appropriate environment variables. The updated image is built automatically when running `docker compose up --build`.

8.4. Write Permission to Gold Layer

Challenge: Airflow container could not write to mounted `datalake_gold/` directory due to UID mismatches.

Solution: Added a one-time initialization step that runs `chown -R 50000:0 /opt/airflow/datala` to set ownership to the Airflow container user.

8.5. Inconsistent Timestamp Formats

Challenge: Multiple timestamp formats across sources (Twitter string format, numeric milliseconds, ISO strings).

Solution: Implemented `normalize_event_date()` function that detects format via pattern matching and coercion, as documented in Section 2.

9. Conclusions and Next Steps

9.1. Summary of Achievements

Workshop #3 has successfully delivered the Gold layer implementation for the Tech Products & Brand Perception sentiment analysis project:

1. **Pipeline Improvements**: Systematic identification and resolution of five critical issues (Parquet compatibility, UDF serialization, date parsing, Docker permissions, Java dependency) that prevented reliable Gold layer processing.

2. **Silver-to-Gold PySpark DAG**: A fully functional Airflow DAG that reads all Parquet files from `datalake_silver/` using distributed processing, applies transformations, and writes timestamped outputs to `datalake_gold/`.

3. **Governance KPIs**: Seven data quality metrics defined and computed, each justified by actual observations from the Silver layer data quality findings.

4. **Storytelling Aggregations**: Six analytical summaries mapped directly to user stories from Workshop #1, forming a coherent narrative about technology product sentiment.

5. **Reproducible Evidence**: Both governance and storytelling Parquet files generated successfully, with documented execution logs and data samples.

9.2. Feasibility Assessment Update

The project remains highly feasible with the completed Gold layer:

- **Data Accessibility**: Both sources continue to provide reliable data through API and web scraping.
- **Pipeline Robustness**: The resilient reader and fallback mechanisms handle edge cases without failure.
- **Governance Visibility**: KPIs provide clear visibility into data quality, enabling proactive issue detection.
- **Analytical Value**: Storytelling aggregations directly support the product marketing analyst's decision-making needs.

9.3. Next Steps for Workshop #4

The foundation established in this workshop enables the following activities for Workshop #4:

1. **Plotly Dash Dashboard Implementation**: Build an interactive dashboard that reads exclusively from the Gold Parquet files created in this workshop.

2. **Visualization Design**: Create charts and KPI cards for each storytelling aggregation: sentiment trend lines, keyword bar charts, source comparison donut charts, and volume heatmaps.

3. **Dashboard Interactivity**: Implement filters for date ranges, sources, sentiment categories, and product brands.

4. **Deployment**: Deploy the dashboard as a standalone application or integrated with the Airflow environment.

5. **User Acceptance Testing**: Present the dashboard to the functional user (product marketing analyst) and incorporate feedback.

9.4. Final Remarks

This workshop demonstrated the value of iterative improvement based on real experimentation findings. The initial Silver pipeline design, while functional, did not anticipate several data quality challenges (mixed Parquet encodings, UDF serialization requirements, multi-format timestamps). By systematically identifying and addressing these issues, we have produced a robust Gold layer that provides both data governance visibility and actionable analytical insights.

The combination of PySpark for distributed processing and Parquet for columnar storage positions the pipeline well for potential scaling to larger datasets, while the modular DAG design ensures maintainability for future enhancements.

10. References

- Apache Airflow Documentation. (2026). https://airflow.apache.org/docs/
- Apache Spark Documentation. (2026). https://spark.apache.org/docs/latest/
- Pandas Documentation. (2026). https://pandas.pydata.org/docs/
- PyArrow Documentation. (2026). https://arrow.apache.org/docs/python/
- GSMArena. (2026). https://www.gsmarena.com
- X/Twitter API via RapidAPI. (2026). https://rapidapi.com
- Docker Compose Documentation. (2026). https://docs.docker.com/compose/
- Plotly Dash Documentation. (2026). https://dash.plotly.com/

11. Appendix: Repository Structure

```
project_root/
 airflow/
    dags/
       bronze_ingestion_dag.py
       silver_processing_dag.py
       gold_processing_dag.py
       gold_helpers.py
    Dockerfile
    docker-compose.yml
    requirements.txt
 datalake_bronze/
    twitter_tweets_*.json
    webscraping_noticias_*.json
 datalake_silver/
    twitter_processed_*.parquet
    gsmarena_processed_*.parquet
 datalake_gold/
    governance_YYYYMMDD_HHMMSS.parquet
    storytelling_YYYYMMDD_HHMMSS.parquet
 notebooks/
    data_quality_findings.ipynb
    gold_preview.ipynb
 workshop_1/
    Workshop_1_Document.pdf
 workshop_2/
    Workshop_2_Document.pdf
 workshop_3/
    gold_pipeline_report.pdf
 README.md
```